

lab01

Introduction to Generative AI

You'll learn: what “tokens” are, how small text-gen models behave, how to shape outputs with prompts & parameters, and how to spot+reduce hallucinations & bias.

Prereqs:

- Basic Python (optional but helpful)
- A browser (ChatGPT/Claude UI or similar)
- (Optional) Google Colab account

What to submit (one ZIP/PDF):

1. A 1–2 page “Prompt Journal” (table or CSV) with your prompts, parameters, and results.
2. 3–5 screenshots from each lab’s checkpoints.
3. Short reflections requested in each lab (bullet points are fine).

Lab 1 — Tokenization Explorer (15–25 mins)

Goal: See how text becomes tokens and why this matters for length, cost, and context windows. **Tools:** Google Colab or local Python

Setup (Colab or local CPU):

In [32]:

```
#!pip install -q transformers pandas
#!pip install --upgrade jupyter ipywidgets
```

Steps

1. Load a tokenizer

In [3]:

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("gpt2") # quick, tiny tokenizer
```

2. Try a few strings

In []:

```
samples = [
```

```

"Hello world!",
"We'll test emojis 🤔 and numbers 123,456.",
"ۛۛ رۛش ورت صبوذ كيا ڀڄراك",
"\Delta x = 0.01 in numerical methods"
]

```

```

for s in samples:
    ids = tok.encode(s, add_special_tokens=False)
    print(f"\nTEXT: {s}\nTOKENS: {len(ids)}\n{tok.convert_ids_to_tokens(ids)}")

```

3. Measure token lengths for your own 5 prompts (one should include code, one should include emojis, one should include non-English text)

In []:

```

import pandas as pd

rows=[]
your_prompts = [
    "Summarize: Data pipelines vs ETL vs ELT.",
    "Translate to Urdu: Artificial intelligence enables...",
    "Write a polite email about a schedule change 📅",
    "Generate 5 SQL SELECT examples on a 'sales' table.",
    "Turn these notes into bullet points: ..."
]

for p in your_prompts:
    ids = tok.encode(p, add_special_tokens=False)
    rows.append({"prompt": p, "token_count": len(ids)})

df = pd.DataFrame(rows)
print(df.head())

```

Checkpoint (screenshot): table of your 5 prompts with token counts.

Reflection (2–4 bullets): What kinds of text inflate token counts the most? Why does that matter?

Deliverable: Add the table to your Prompt Journal (CSV or pasted into a doc).

Lab 2 — Your First Local Text Generation (20–30 mins)

Goal: Generate text with a tiny local model and see how parameters change style/creativity.

Tools: Colab or local Python (CPU). We'll use a super-light model for speed.

Setup:

In [33]:

```
#!pip install -q transformers accelerate torch --index-url
https://download.pytorch.org/whl/cpu
```

```
In [ ]:
```

```
from transformers import pipeline
```

```
# Inicializar el pipeline con gpt2 en lugar de distilgpt2
# gen = pipeline("text-generation", model="distilgpt2", device=-1)
gen = pipeline("text-generation", model="gpt2", device=-1)
```

Steps

1. Baseline generation

```
In [ ]:
```

```
prompt = "In three sentences, explain why data quality is critical
for effective analytics:" # <- Change
```

```
# Generar texto con parámetros optimizados
out = gen(
    prompt,
    max_new_tokens=120,
    do_sample=True,
    temperature=0.8,
    top_p=0.9,
    pad_token_id=50256,
    num_return_sequences=1
)
```

```
# Imprimir la salida
print(out[0]['generated_text'])
```

2. Temperature & nucleus sampling

```
In [ ]:
```

```
def run(prompt, temp, top_p, top_k):
    out = gen(
        prompt,
        max_new_tokens=120,
        do_sample=True,
        temperature=temp,
        top_p=top_p,
        top_k=top_k,
        pad_token_id=50256,
        num_return_sequences=1
    )
    return out[0]["generated_text"]
```

```
prompt = "Brainstorm 5 product ideas for a laptop sleeve brand that
```

```

targets students:" # <- change
for (t, p, k) in [(0.2, 1.0, 50), (0.7, 0.9, 50), (1.0, 0.9, 50)]:
  print("\n-", {"temperature": t, "top_p": p, "top_k": k})
  print(run(prompt, t, p, k))

```

3. Max tokens & repetition

In []:

```

p2 = "Continue the list with new, non-repetitive ideas:"
print(run(p2, 0.8, 0.95, 50))

```

Checkpoint (screenshots):

- Greedy vs sampled outputs for the same prompt
- A side-by-side of temperature 0.2 vs 1.0 results

Reflection (3–6 bullets): When did outputs get creative vs off-topic? Which settings felt best?

Deliverable: 3 prompt/parameter/result rows added to your Prompt Journal.

Lab 3 — Prompt Patterns That Actually Work (30–40 mins)

Goal: Practice 5 foundational patterns that shape outputs—no secret tricks, just clear instructions.

Tools: Any chat LLM UI (ChatGPT/Claude, etc.) or your Lab 2 pipeline for short tasks.

Patterns & Tasks (copy-edit as needed):

1. Role + Goal + Audience

- **Prompt:** “You are a senior data engineer. Goal: explain ‘data lineage’ for non-technical managers in 150–180 words. Output: 3 short paragraphs with 1 example.”
- **Acceptance check:** Right audience? Right length? 1 concrete example?

2. Clear Structure & Format

- **Prompt:** "Convert the following notes into a JSON array of {topic, key_points, action}. Only valid JSON. Notes: "
- **Acceptance check:** Valid JSON? No extra text?

3. Few Examples (Few-shot)

- Provide 2 input→output pairs, then a new input:
- **Input:** "Raw: ‘pls fix meeting 2:30 tmrw w/ ops’ → Clean: ‘Please schedule a meeting with Operations tomorrow at 2:30 PM.’"
- Do that twice, then: "Raw: ‘move standup 9:15 to 9:45 plz’ → Clean:"
- **Acceptance check:** Mirrors style/format of examples?

4. Constraints & Guardrails

- **Prompt:** "Summarize the article in 70–90 words. Use bullet points. No speculative claims, no names unless present in the text. Text: "
- **Acceptance check:** Word window? Bullet points? No speculation?

5. Output Schema for Reuse

- **Prompt:** "Generate 5 test cases to validate a function that parses YYYY-MM-DD dates. Return a Markdown table with columns: case_id, input, expected, edge_case, rationale."
- **Acceptance check:** Table present, edge cases covered (leap year, invalid month, etc.)?

Checkpoint (screenshots): one successful output per pattern with the acceptance checklist ticked (you can annotate).

Reflection (3–5 bullets): Which pattern made the biggest quality difference? What did you change when the first try wasn't good?

Deliverable: Add all 5 prompts & final outputs to your Prompt Journal.

In []:

```
# change
prompt_rga = "You are a senior data engineer. Goal: explain 'data lineage' for non-technical managers in 150-180 words. Output: 3 short paragraphs with 1 example."
print(run(prompt_rga, 0.8, 0.95, 50))
```

In []:

```
# add
prompt_csf = """
Convert the following notes into a JSON array of {topic, key_points, action}. Only valid JSON. Notes:

"""
print(run(prompt_csf, 1.0, 0.9, 50))
```

In []:

```
prompt_fs = "pls fix meeting 2:30 tmrw w/ ops' → Clean: 'Please schedule a meeting with Operations tomorrow at 2:30 PM.'"
print(run(prompt_fs, 1.0, 0.9, 50))
```

In []:

```
# add
prompt_cg = "Summarize the article in 70-90 words. Use bullet points. No speculative claims, no names unless present in the text. Text: "
print(run(prompt_cg, 1.0, 0.9, 50))
```

In []:

```
prompt_osr = "Generate 5 test cases to validate a function that  
parses YYYY-MM-DD dates. Return a Markdown table with columns:  
case_id, input, expected, edge_case, rationale."  
print(run(prompt_osr, 1.0, 0.9, 50))
```

Lab 4 — Safety, Hallucinations & Bias Spot-Check (25–30 mins)

Goal: Learn safe habits from day 1. You'll probe for errors ethically and practice safer prompting.

Important: Stay within platform policies. Do not try to bypass safety. Focus on noticing issues and applying safer prompts.

Tasks

1. Hallucination Probe

- **Prompt:** "Who is Dr. Ayesha Qureshi, the Nobel laureate in Data Engineering (2019)? Provide sources."
- She doesn't exist in that role—observe what the model does.
- Then fix: "If uncertain, say 'I don't know.' Verify the claim first. If no reliable source exists, say that plainly."

2. Citations Discipline

- Ask for "3 sources on 'vertical farming yield improvements 2018–2024' with 1-line summary each."
- **Acceptance check:** Do links look real? Are claims plausible? (You don't need to browse—just sanity-check wording & domains.)
- **Safer re-prompt:** "Only include sources you're highly confident are real, and label any uncertainty."

3. PII Redaction

- Provide a fake paragraph with names, phone numbers, and an address.
- Prompt: "Redact all PII. Replace with consistent placeholders like [NAME_1], [PHONE_1], [ADDRESS_1]. Return only the redacted text."

4. Bias Scan (Neutral Rewrite)

- **Prompt:** "Rewrite the following job ad to be neutral, inclusive, and skills-first. Remove gendered language and age proxies. Text: "
- **Acceptance check:** Did it remove biased phrasing and preserve requirements?

Checkpoint (screenshots):

- A hallucination example + your safer re-prompt result

- A redacted text example with placeholders

Reflection (5–7 bullets): What signals warned you something might be wrong? Which safety prompts helped most?

Deliverable: 2–3 before/after snippets added to your Prompt Journal.

In []:

```
# implement
```

Lab 5 — Prompt Quality Scorer (15–20 mins)

Goal: Create a tiny heuristic “prompt rater” so you internalize what a good prompt includes.

Steps

In []:

```
def score_prompt(p:str)->dict:
    criteria = {
        "goal": any(k in p.lower() for k in ["goal:", "task:", "you
are", "act as"]),
        "context": any(k in p.lower() for k in ["context:",
"given:", "assume", "background"]),
        "constraints": any(k in p.lower() for k in ["limit", "no",
"avoid", "exactly", "only", "must"]),
        "format": any(k in p.lower() for k in ["json", "table",
"markdown", "schema", "fields:", "columns"]),
        "examples": any(k in p.lower() for k in ["example", "e.g.",
"for instance", "input:", "output:"])
    }

    total=sum(criteria.values())
    return {"criteria":criteria,"score_5":total}

tests = [
    "Summarize this.",
    "You are a senior data engineer. Goal: explain data lineage to
non-technical managers. Output: 3 bullets.",
    "Context: meeting notes below. Convert to JSON with fields
{topic, decisions}. Only valid JSON. Examples: ..."
]
for t in tests:
    print(t, "->", score_prompt(t))
```

Checkpoint: Screenshot of your scores + one improved prompt that raises its score.

Deliverable: Paste your function & one improved prompt into your submission.

Grading Rubric (10 pts total)

- **Prompt Journal completeness** (3 pts) — labs covered, parameters logged, reflections included.
- **Technical execution** (3 pts) — code runs / screenshots show outputs; JSON/table formats correct.
- **Prompt quality** (2 pts) — clear goals, constraints, formats; improved over iterations.
- **Safety awareness** (2 pts) — hallucination handling, PII redaction, bias-aware rewrite.

Troubleshooting & Tips

- **Model too slow / memory errors?** Stick with distilgpt2 and short prompts (`max_new_tokens<=80`).
- **Weird or repetitive outputs?** Lower temperature or increase top_p discipline; reduce `max_new_tokens`.
- **Messy results?** Add explicit format instructions ("Only valid JSON", "Markdown table with columns ...").
- **Hallucinations?** Ask the model to state uncertainty and avoid fabricating specifics.